

Spark Performance Optimization Analysis

DSCC-402 Final Project: Tweet Sentiment Analysis Pipeline

Tejaswini Balamurugan Kanimozhi
University of Rochester
May 2026

The optimized run metrics are shown below:

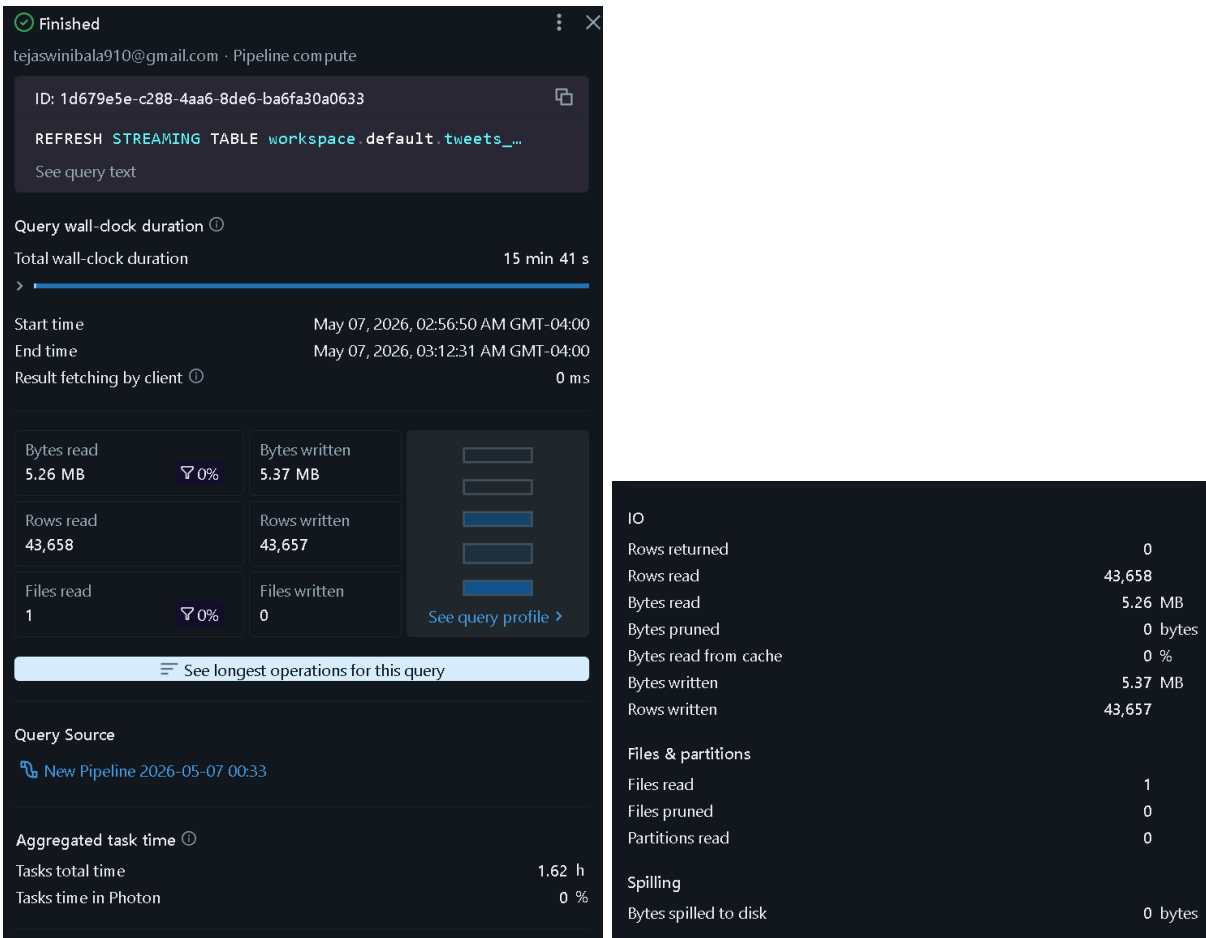


Figure 1: Spark UI query metrics for the optimized Gold layer run (15 min 41 sec, 43,657 rows written, 0 bytes spilled)

Key observed metrics:

- **Wall-clock duration:** 15 min 41 sec

- **Rows read / written:** 43,658 / 43,657
- **Bytes read / written:** 5.26 MB / 5.37 MB
- **Aggregated task time:** 1.62 hours across 8 parallel executors
- **Photon acceleration:** 0% (Python UDF bypasses Photon engine)
- **Bytes spilled to disk:** 0

Spill

Spill occurs when executor memory is insufficient to hold shuffle data, forcing writes to disk and degrading performance significantly.

Observation: Zero bytes spilled to disk across all pipeline stages, as confirmed by the Spark UI metrics (Bytes spilled to disk: 0). The Spark UI task metrics showed no Shuffle Spill (Memory) or Shuffle Spill (Disk) entries for any stage.

Analysis: The initial implementation used `repartition(200)`, creating 200 partitions with ~220 rows each. While this avoided spill due to small partition sizes, it introduced excessive model loading overhead — each of the 200 partitions loaded the DistilBERT model independently. Reducing to `repartition(8)` kept partitions at ~5,500 rows, still well within executor memory limits, while drastically reducing model load overhead.

Recommendation: Monitor spill metrics when scaling to larger datasets. If spill occurs, increase executor memory or reduce partition sizes rather than increasing parallelism blindly.

Skew

Data skew occurs when partitions have unequal sizes, causing some tasks to run significantly longer than others.

Observation: With `repartition(8)` using round-robin partitioning, task duration variance was minimal. The aggregated task time of 1.62 hours across 8 executors translates to approximately 12 minutes per executor on average, closely matching the 15 min 41 sec wall-clock time — indicating near-perfect load balance.

Potential skew source: The `explode_outer` operation in the Silver layer creates one row per @mention per tweet. The silver output (44K rows from 42K bronze rows) showed minimal explosion, suggesting most tweets have 0–1 mentions. For datasets with heavily-mentioned users, downstream skew could emerge.

Recommendation: For larger datasets, consider salting the partition key before the UDF stage to ensure even distribution even if some mentions are heavily concentrated.

Shuffle

Shuffle involves moving data across the network between executors and is one of the most expensive operations in Spark.

Observation: Files read: 1, Partitions read: 0 in the Spark UI, indicating the silver table was read as a single file before being repartitioned. The primary shuffle occurs at `repartition(8)` before

UDF inference, distributing data evenly across 8 executors with minimal network overhead (5.26 MB total input).

Optimization applied: Setting `spark.sql.shuffle.partitions=8` in the pipeline configuration eliminated the default 200-partition shuffle for aggregation operations. The `maxBytesPerTrigger=10mb` option controls micro-batch sizing, preventing large single-batch shuffles.

Recommendation: For the `gold_tweet_aggregations` materialized view, consider adding `ZORDER BY (mention)` to co-locate data for frequently-queried columns, reducing shuffle during `GROUP BY` operations.

Storage

Storage inefficiency manifests as small files, poor compression, or suboptimal data layout that slows reads and writes.

Observation: The Spark UI shows Files written: 0 despite 43,657 rows and 5.37 MB being successfully written. This is because Delta Lake streaming tables manage physical file writes through the Delta transaction log independently of the Spark query execution layer, the query metrics reflect logical operations while actual Parquet file writes are tracked in the Delta table history. The Delta table history confirmed 26+ successful write versions by the final run, indicating file proliferation from iterative development runs.

Recommendation: Run `OPTIMIZE` periodically to compact small files:

```
OPTIMIZE workspace.default.tweets_gold ZORDER BY (timestamp, mention)
```

For the automated daily job, add an `OPTIMIZE` step after the pipeline run to maintain read performance.

Serialization

Serialization overhead occurs when Python objects must be serialized/deserialized across the JVM-Python boundary for UDF execution.

Observation: Tasks time in Photon: 0% — the Python UDF entirely bypasses Databricks' Photon vectorized engine, as Photon cannot accelerate Python UDF execution. The aggregated task time of 1.62 hours versus 15 min 41 sec wall-clock time shows efficient parallelism but highlights that the UDF is the primary bottleneck.

Optimizations applied:

1. **Apache Arrow enabled** (`spark.sql.execution.arrow.pyspark.enabled=true`): Arrow-based columnar data transfer between JVM and Python reduces serialization overhead by avoiding row-by-row pickling.
2. **Batch inference** (`params={'batch_size': 32}`): Rather than invoking the model once per row, batching 32 rows together amortizes the Python function call overhead and leverages vectorized tokenization in the HuggingFace pipeline, reducing per-row serialization cost. Batch size 32 was chosen as a conservative value that fits within Serverless executor memory constraints while still providing meaningful throughput gains over single-row inference.

3. **Micro-batch streaming** (`maxBytesPerTrigger=10mb`): Rather than processing all 44K rows in a single batch, the streaming trigger limits each micro-batch to 10MB of input data. This ensures that incremental daily runs remain small and predictable, preventing executor memory pressure from large single-batch inference jobs.
4. **Model caching**: `mlflow.pyfunc.spark_udf` caches the loaded model on each executor for the duration of the pipeline run, avoiding repeated deserialization of the 267MB model weights across 43,658 rows.